

Vibecoding Workshop: Build a Nigerian Legal Search Tool

A step-by-step guide for lawyers with zero coding experience, using GitHub Copilot in VS Code

What you'll build: a working web app where you can search a small library of Nigerian legal documents (Constitution, Labour Act, CAMA, etc.) and get back the most relevant excerpts — optionally with an AI-generated plain-English summary that cites its sources.

How this guide works: every grey box is text you copy and paste, either into a terminal or into Copilot Chat. You do not need to understand the code it produces. Your job is to paste each prompt **one at a time**, wait for it to finish, do the quick "Verify" check underneath, and only then move to the next one. If a step fails, don't panic and don't skip ahead — just say so, and your facilitator (or Copilot itself) will help you fix it before continuing.

Part 0 — Install the software (do this before the session if possible)

You need four things on your laptop: VS Code, Node.js, Git, and the GitHub Copilot extension. This takes about 20 minutes.

0.1 Install VS Code

1. Go to <https://code.visualstudio.com>
2. Click the big **Download** button for your operating system (Windows/Mac/Linux).
3. Open the downloaded file and click through the installer with default options.
4. Open VS Code once to confirm it launches.

0.2 Install Node.js

This is the engine that runs the app on your computer.

1. Go to <https://nodejs.org>
2. Download the version labelled **LTS** (Long-Term Support) — not "Current."
3. Run the installer with default options, clicking "Next" through all screens.
4. To confirm it worked: open VS Code, then open its built-in terminal via the menu **Terminal** → **New Terminal** (or `Ctrl+`` on Windows/Linux, `Cmd+`` on Mac). Type:

```
node -v
```

and press Enter. You should see something like `v22.x.x`. If you see "command not found," restart your computer and try again — this fixes it almost every time.

0.3 Install Git

1. Go to <https://git-scm.com/downloads>
2. Download and install for your OS with default options.
3. In the VS Code terminal, confirm with:

```
git --version
```

0.4 Create a GitHub account (if you don't have one)

Go to <https://github.com> and sign up. This is where your project's code will live and how it connects to Copilot and to deployment.

0.5 Install GitHub Copilot in VS Code

1. In VS Code, click the **Extensions** icon in the left sidebar (looks like four squares).
2. Search for **GitHub Copilot**.

3. Click **Install** on the extension published by GitHub. This will also install "GitHub Copilot Chat" alongside it.
4. VS Code will prompt you to sign in to GitHub — click **Sign in**, and authorize in the browser tab that opens.
5. Copilot requires an active subscription (GitHub offers a free trial for individual accounts, and free tiers change over time). If you're not sure whether your account has access, check the current plans at <https://github.com/features/copilot> — your facilitator can also help confirm before the session.

0.6 Open the Copilot Chat panel and switch to Agent mode

This is the panel you'll paste every prompt into for the rest of this guide.

1. Press **Ctrl+Shift+I** (Windows/Linux) or **Cmd+Shift+I** (Mac) to open the Copilot Chat panel. (If that doesn't work, click the Copilot icon in the top toolbar or the sidebar.)
2. At the top of the chat panel there's a mode dropdown — it usually says **Ask**, **Edit**, or **Agent**. Click it and select **Agent**.
 - **Ask** mode only answers questions — it won't touch your files. **Agent** mode can create files,

edit code, and run terminal commands on your behalf, which is what we want for this workshop.

3. Agent mode will ask for your permission before running most terminal commands. Always read what it's about to run and click **Run** (or **Continue**) to approve it — don't approve blindly, but for this workshop the commands are all safe and expected.

Verify before moving on: you should have a chat panel open on the right (or bottom) of VS Code, with "Agent" selected in the mode dropdown.

Part 1 — Set up your free accounts (~15 min)

You need three free accounts before you start building. Do these now so you're not waiting on emails/verification during the session.

1.1 Upstash (your document database)

1. Go to <https://upstash.com> and sign up (you can use your GitHub account to sign up in one click).
2. Once logged in, click **Create Database**, then choose **Vector** (not Redis/Kafka).

3. Give it a name like `naija-legal-search`.
4. When asked about the embedding model, choose an option **with a built-in embedding model** — look for something like `mxbai-embed-large-v1` in the dropdown. This matters: it means you can send plain text straight to the database, without needing a separate step to convert text into number-vectors yourself.
5. Once created, open the database and find the **REST API** section. Copy these two values somewhere safe (a Notes app, a text file) — you'll need them shortly:
 - `UPSTASH_VECTOR_REST_URL`
 - `UPSTASH_VECTOR_REST_TOKEN`

1.2 Vercel (where your app will live on the internet)

1. Go to <https://vercel.com> and sign up **using your GitHub account** — this makes deployment later a one-click process.

1.3 Google Gemini API key (only needed if you're doing the optional AI-summary feature)

1. Go to <https://aistudio.google.com/app/apikey>
2. Sign in with a Google account.
3. Click **Create API key**, and select or create a Google Cloud project when prompted (the default

option is fine).

4. Copy the key somewhere safe. Treat it like a password – don't paste it into chat, don't share it publicly, don't commit it to GitHub.

Verify before moving on: you should have three things saved in your notes: the two Upstash values, and the Gemini API key.

Part 2 – Source your legal documents (~20–30 min)

The AI must **never** be allowed to invent legal text from memory – that's the core lesson of this workshop. Instead, we give it a small, real library of documents to search over.

1. Go to <https://nigerialii.org> (Nigerian Legal Information Institute – free access to Nigerian primary law).
2. Copy **8–15 short excerpts**. A good mix for a demo:
 - 3–4 sections of the 1999 Constitution
 - 2–3 sections of the Labour Act
 - 2–3 sections of CAMA or the Evidence Act
 - 1–2 short, plain-English case summaries that **you write yourself** (don't paste full judgments

— a few sentences summarizing the holding is enough)

3. Paste each excerpt into its own plain text file using VS Code:

- Click **File** → **New Text File**, paste the excerpt, then save it (Ctrl+S / Cmd+S) with a clear name like `labour-act-section-11.md`, into a folder on your Desktop for now (e.g. a folder called `legal-docs-source`). You'll move these into the project itself in Part 4.

4. Keep each file short — a section or two of a statute, not an entire Act. 8 short documents is enough to demonstrate real, convincing retrieval.

Verify before moving on: you have a folder with 8–15 small `.md` or `.txt` files, each containing one real legal excerpt.

Part 3 — Build the project

From here on, every grey box is a prompt. **Paste it into the Copilot Chat panel (Agent mode), press Enter, let it finish completely, do the Verify step, then move to the next prompt.** Don't paste two prompts in a row without checking the first worked — small errors compound quickly in AI-driven builds.

If Copilot asks to run a terminal command, read it and click **Run/Continue** to approve.

Prompt 1 — Scaffold the project

First, in VS Code, open a folder where you want the project to live: **File** → **Open Folder**, and create/select an empty folder called `naija-legal-search`. Then open the Copilot Chat panel and paste:

```
Create a new Next.js 14+ project using the
App Router and TypeScript, in this
current folder, called "naija-legal-
search". Set up Tailwind CSS. Initialize
git. Check the latest official Next.js
documentation for the correct
create-next-app flags before running the
command, since I want current best
practice, not outdated syntax.
```

Verify: in the terminal, type `npm run dev`, press Enter, then open the link it prints (usually `http://localhost:3000`) in your browser. You should see the default Next.js starter page. Stop the server afterward with `Ctrl+C` in the terminal.

Prompt 2 — Install and wire up dependencies

```
Install the following packages:
@upstash/vector, ai (Vercel AI SDK),
@ai-sdk/google. Before writing any code,
```

```
check the current official
documentation for @upstash/vector to
confirm the correct method names and
syntax for upserting and querying using an
index with a built-in embedding
model (i.e., passing raw text via a "data"
field rather than a precomputed
"vector" field) – I want this to match the
current SDK, not an assumed API.
Create a .env.local.example file with
placeholders for
UPSTASH_VECTOR_REST_URL,
UPSTASH_VECTOR_REST_TOKEN, and
GOOGLE_GENERATIVE_AI_API_KEY (this is the
exact variable name @ai-sdk/google
expects by default).
```

Verify: in the VS Code file explorer (left sidebar), open `.env.local.example` and confirm it lists those three variable names.

Now create your **real** secrets file, which never gets uploaded to GitHub:

1. In the file explorer, right-click the project's root folder → **New File** → name it exactly `.env.local`
2. Paste in your real values from Part 1:

```
UPSTASH_VECTOR_REST_URL=paste-your-
value-here
UPSTASH_VECTOR_REST_TOKEN=paste-your-
value-here
```

```
GOOGLE_GENERATIVE_AI_API_KEY=paste-  
your-value-here
```

3. Save the file.

Prompt 3 – Add your legal document corpus

```
Create a /data folder. I will paste in [N]  
legal document files. For each  
one, add appropriate frontmatter metadata  
at the top: title, source, and  
category (constitution / statute / case-  
summary). Don't alter the legal  
text itself – only add the metadata  
wrapper.
```

Replace [N] with the actual number of documents you sourced in Part 2. Then, one at a time (or all at once if Copilot allows it), paste in the content of each file you saved earlier, telling Copilot which file it belongs to.

Verify: you should now see a /data folder in the file explorer containing your files, each with a title: / source: / category: block at the top.

Prompt 4 – Ingestion script (loads your documents into the database)

```
Write a script at /scripts/ingest.ts that:
1. Reads every file in /data
2. Splits any document longer than ~500
words into overlapping chunks
   (~500 words, ~50 word overlap)
3. Upserts each chunk into my Upstash
Vector index using the built-in
   embedding model (raw text upsert, not
precomputed vectors), including
   metadata: title, source, category, chunk
index
4. Logs progress to the console as it runs,
and prints a final count of
   chunks ingested
Use the Upstash Vector syntax you confirmed
in the previous step.
```

Verify: in the terminal, run the command Copilot suggests (usually `npx tsx scripts/ingest.ts`). Watch it print progress and a final chunk count — that count should roughly match your number of documents (a bit higher if any were split into chunks). If you see an authentication error, double check your `.env.local` values were copied correctly with no extra spaces.

Prompt 5 — Sanity-check retrieval before building the UI

```
Write a tiny throwaway test script that
queries the Upstash Vector index
```

```
with the text "employee termination notice"
and prints the top 3 results
with their titles and scores, so I can
sanity-check retrieval quality
before building the UI.
```

Verify: run the script it suggests. The top results should actually be about termination/notice-type content. If the results look random or irrelevant, tell Copilot — it's usually a chunking or ingestion issue, and it's worth fixing before continuing.

Prompt 6 — Search API route

```
Create app/api/search/route.ts as a POST
endpoint. It accepts a JSON body
{ query: string }, queries the Upstash
Vector index for the top 5 most
similar chunks, and returns a JSON array of
{ title, source, snippet, score }.
Add basic validation: reject empty queries
with a 400 and a clear error
message. Add a try/catch that returns a 500
with a readable error message
on failure (this app will be used live in
front of an audience, so failures
need to be visible and understandable, not
silent).
```

Verify: ask Copilot Chat directly: *"Give me a curl command to test the /api/search route with the query*

'employee rights', and run it for me." Let it run in the terminal — you should see JSON results printed back.

Prompt 7 — Search UI (the screen you'll actually demo)

```
Build app/page.tsx as a clean, professional
legal-research search interface:
- A prominent search input and submit
button
- A loading state while the request is in
flight
- Results displayed as cards: document
title, source/category badge,
  matching snippet with the query terms
  subtly highlighted if easy to do,
  and relevance score
- A friendly empty state before the first
search ("Search Nigerian legal
  documents — try 'employee termination
  notice' or 'company registration
  requirements'")
- A clear empty-results state ("No matching
documents found — try
  rephrasing your query")
Use Tailwind. Style it like a professional
legal research tool — clean,
minimal, trustworthy, not playful. Make it
responsive for a projector/laptop
demo.
```

Verify: run `npm run dev` again, open `http://localhost:3000` in your browser, and try 2–3 real searches end-to-end. This is your first fully working demo – take a moment to enjoy it.

Prompt 8 – Optional stretch goal: AI-synthesized answer with citations

Only do this if you have time and a Gemini API key set up. This is also the step your facilitator will use to discuss hallucination risk, so read the Verify step carefully.

```
Add an optional "Get AI Summary" button
that appears after search results
load. When clicked, it sends the top 3
retrieved chunks plus the user's
original question to Gemini via the AI SDK
(@ai-sdk/google, using the
GOOGLE_GENERATIVE_AI_API_KEY env var and a
current Gemini model – check
the current @ai-sdk/google docs for the
correct model identifier string),
asking it to produce a short, plain-English
answer that explicitly cites
which source document each part of the
answer draws from. Prominently
display a disclaimer beneath the AI answer:
"This is a general information
summary generated from the documents above,
not legal advice. Verify all
citations independently." Stream the
```

```
response if straightforward,  
otherwise a simple loading state is fine.
```

Verify: click the button after a search. Confirm the disclaimer is clearly visible, and — importantly — spot-check that the citations in the AI's answer actually match the documents that were retrieved. If it cites a source that wasn't in your top 3 results, that's a hallucination, and it's worth flagging to the group rather than hiding it.

Prompt 9 — Error handling and polish

```
Do a review pass across the whole app:  
handle the case where  
environment variables are missing (show a  
clear setup error rather than  
a crash), handle network failures  
gracefully, and make sure nothing  
throws an unhandled error in the browser  
console during normal use.  
Add a simple footer noting this is a  
workshop demo app.
```

Prompt 10 — README and one-click deploy button

First, create a GitHub repository for your project:

1. Go to <https://github.com/new>, name it `naija-legal-search`, leave it public or private as you

prefer, and click **Create repository** (don't initialize with a README — you already have a project).

2. Back in Copilot Chat, paste:

```
Push this project to the GitHub repository at https://github.com/YOUR-USERNAME/naija-legal-search (ask me to confirm the exact URL before running any git remote commands).
```

Confirm your exact repo URL when it asks, and approve the git commands it proposes.

Then paste:

```
Write a README.md that explains what this app does, lists the required environment variables, gives setup steps (clone, npm install, copy env file, run ingest script, npm run dev), and includes a Vercel "Deploy" button (deploy button markdown pointing to this GitHub repo) so students can fork and deploy it themselves with one click during the workshop.
```

Verify: refresh your GitHub repository page in the browser and confirm your code and README are there.

Part 4 — Deploy it to the internet (~15 min, done manually — not via Copilot)

1. Go to <https://vercel.com>, click **Add New Project**, and import your `naija-legal-search` GitHub repository.
2. On the configuration screen, expand **Environment Variables** and add all three:
 - `UPSTASH_VECTOR_REST_URL`
 - `UPSTASH_VECTOR_REST_TOKEN`
 - `GOOGLE_GENERATIVE_AI_API_KEY` (if you built the AI-summary step)
3. Click **Deploy**. Wait for the build to finish — Vercel will give you a live URL (something like `naija-legal-search.vercel.app`).
4. **Open the live URL in your browser and test it there**, not just on `localhost`. Environment variable mistakes often only show up once it's live.

Troubleshooting cheat sheet

Problem	Likely fix
<code>node -v</code> says "command not found"	Restart your computer after installing Node.js, then try again.
Copilot Chat panel won't open	Make sure the GitHub Copilot Chat extension installed (Part 0.5), and that you're signed in to GitHub in VS Code (bottom-left account icon).
Agent mode isn't in the dropdown	Your GitHub account may not have an active Copilot subscription/trial – check https://github.com/features/copilot , or ask your facilitator.
Ingestion script errors with "unauthorized"	Double-check <code>.env.local</code> – no extra spaces, no quote marks around the values, correct variable names.
Search results look irrelevant	Usually a chunking issue – tell Copilot directly what's wrong ("the results for X don't seem relevant, can you check the chunking and query logic") and let it investigate.
Live Vercel site shows an	You likely forgot to add one of the environment variables in the Vercel

Problem	Likely fix
error the local one didn't	project settings — go back and check.

A note on what this workshop is really teaching

This app is small on purpose. The real lesson isn't "how to build a search tool" — it's what it feels like to **watch retrieval work correctly**, so you can recognize what it looks like when an AI tool *isn't* actually grounding its answers in real source documents. Every time you paste a query and see a real, citable excerpt come back, that's the behavior you want from any legal AI tool you evaluate in your own practice. Every time the optional AI summary step cites something it shouldn't, that's the failure mode to watch for.